

BFS

```

from collections import deque

# Graph
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['G', 'H'],
    'D': [],
    'E': ['F'],
    'F': [],
    'G': ['J'],
    'H': [],
    'J': []
}

def bfs(graph, start, goal):
    queue = deque([start])
    visited = set()

    while queue:
        node = queue.popleft()

        if node in visited:
            continue

        print("Visiting:", node)
        visited.add(node)

        # Check goal
        if node == goal:
            print("Goal found:", node)
            return

        # Add neighbours
        for neighbour in graph[node]:
            if neighbour not in visited:
                queue.append(neighbour)

    print("Goal not found")

# Initial node = A
# Goal node = F
bfs(graph, 'A', 'F')

```

```

Visiting: A
Visiting: B
Visiting: C
Visiting: D
Visiting: E
Visiting: G
Visiting: H
Visiting: F
Goal found: F

```

DFS

```

# Graph
graph = {

```

```
graph LR
    A --- B
    A --- C
    B --- D
    B --- E
    C --- G
    C --- H
    D --- F
    E --- F
    G --- J
    H --- J
    J --- F

graph TD
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['G', 'H'],
    'D': [],
    'E': ['F'],
    'F': [],
    'G': ['J'],
    'H': [],
    'J': []
}

def dfs(graph, node, goal, visited):
    # Mark node as visited
    visited.add(node)

    print("Visiting:", node)

    # Check if goal is found
    if node == goal:
        print("Goal found:", node)
        return True

    # Visit each neighbour
    for neighbour in graph[node]:
        if neighbour not in visited:
            if dfs(graph, neighbour, goal, visited):
                return True

    return False

# Initial node = A
# Goal node = F
visited = set()

dfs(graph, 'A', 'F', visited)
```

```
Visiting: A
Visiting: B
Visiting: D
Visiting: E
Visiting: F
Goal found: F
True
```